

ckpttn: Multilevel Circuit Partitioning

Wai-Shing Luk

Abstract—Balanced hypergraph partitioning is a fundamental step in VLSI physical design, and the Fiduccia-Mattheyses (FM) local search, refined inside a multilevel framework, is its workhorse. This paper describes **ckpttn**, an open-source multilevel circuit partitioner built on three principles: certify small instances exactly with a middle-levels Gray-code enumeration, keep the FM machinery but make the gain bookkeeping cheap, and coarsen the hypergraph aggressively with a primal-dual minimum maximal matching and Min-Hash duplicate-net pruning. We state the partitioning problem, derive the FM gain updates and their bucket data structure, present a simpler greedy refiner for comparison, and describe a symmetry-reduced enumeration that visits every balanced bipartition. On the p1, ibm01, and ibm03 benchmarks, multilevel refinement reduces the cut cost by 27-54 percent over flat FM, the greedy refiner is 8-35 times faster but 1.4-2.2 times worse, and the same algorithm behaves consistently across Python, C++, and Rust ports, whose inner loops are all organized around the observation that most gain updates are no-ops. The C++ port partitions the full ISPD98 suite, up to 210,000 modules, in under twenty seconds per instance. Unlike classical tools, **ckpttn** warns explicitly when the balance constraint cannot be met.

Index Terms—hypergraph partitioning, circuit partitioning, Fiduccia-Mattheyses, multilevel refinement, min-cut.

I. INTRODUCTION

Circuit partitioning divides the cells of a netlist into a small number of roughly equal parts while cutting as few connections as possible. It is the first step of the physical-design flow and reappears throughout it: as the driver of partitioning-based placement, as the basis of hierarchical design and multi-FPGA mapping, and as a load-balancing primitive in parallel computing. Because the number of cut nets directly limits routability and timing, even a few percent of cut reduction is worth pursuing.

Partitioning has a four-decade history. The Kernighan-Lin (KL) heuristic introduced pairwise interchange as a local search [1]; Schweikert and Kernighan formalized the hypergraph (net) model still in use [2]; and Fiduccia-Mattheyses (FM) replaced pairwise swaps with single-module moves and a bucket priority queue, making one pass linear in the netlist size [3]. Move-based refinement was then extended to k -way partitions [4], [5], [6], while spectral methods pursued the algebraic alternative [7], [8], [9] and network-flow formulations [10], [11] exploited the max-flow/min-cut theorem. The multilevel paradigm – coarsen, partition, then uncoarsen and refine – made FM scale to hundreds of thousands of modules [12], [13], [14], [15], and the ISPD98 benchmark suite standardized how partitioners

are compared [16]. The resulting arc is surveyed by Alpert and Kahng [17], Papa and Markov [18], and Buluç et al. [19].

This paper describes **ckpttn**, a compact, open-source multilevel partitioner. Its design follows three principles.

1. **Certify small instances exactly.** Once a graph is small enough, a middle-levels Gray-code enumeration visits every balanced bipartition of it, turning the multilevel leaf from a heuristic into a certified optimum – the only one of the three principles that guarantees an optimal solution.
2. **Keep FM, but make it cheap.** The Fiduccia-Mattheyses (FM) local search [3] is retained as the refinement operator, but the gain updates are organized so that the common case – a net that stays uncut after a move – does no work, and the inner loop avoids gratuitous allocation and lookup.
3. **Coarsen aggressively.** Large instances are contracted through a primal-dual *minimum maximal matching*, with duplicate nets merged by a MinHash pre-filter, so that the expensive refinement runs on much smaller graphs.

The same algorithm is implemented in Python, C++, and Rust from one specification, and it emits an explicit warning when the requested balance cannot be achieved – a guarantee that widely used tools do not make.

Work since then has pushed in three directions. Multilevel partitioners have adopted n -level hierarchies, community-aware coarsening, and flow-based refinement, culminating in KaHyPar [20], [21], [22], [23] and its scalable shared-memory successor Mt-KaHyPar [24], [25]; the graph-partitioning system KaHIP explored parallel and evolutionary refinement [26]. Constraint-driven partitioners now target modern physical design: TritonPart handles timing, multi-dimensional balance, and embedding constraints [27], SpecPart and K-SpecPart add supervised spectral embeddings [28], [29], and FPGAPart targets interposer-based multi-die FPGAs [30]. Finally, learning-based partitioning is emerging, with graph neural networks evaluated on VLSI hypergraphs [31]. The same algorithms serve the wider scientific-computing ecosystem through tools such as Mondriaan [32] and Zoltan [33], and a recent survey [34] tracks the whole area.

II. PROBLEM FORMULATION

A circuit is modeled as a weighted hypergraph $H = (V, \mathcal{E})$. The vertices V are the modules (cells and I/O pads) and each hyperedge (net) $e \in \mathcal{E}$ connects two or more vertices.

A *k-way partition* assigns every vertex to one of k blocks $P_1, \dots, P_k$ that are pairwise disjoint and cover V . A net is *cut* if it has pins in more than one block.

The objective is to minimize the total weight of the cut nets,

$$\min \sum_{e \in \mathcal{E}} w(e) \cdot \mathbb{I}[e \text{ is cut}],$$

subject to a balance constraint on the block weights. With $m(v)$ the module weight and $W = \sum_{v \in V} m(v)$ the total weight, each block must satisfy

$$(1 - \epsilon) \frac{W}{k} \leq \sum_{v \in P_i} m(v) \leq (1 + \epsilon) \frac{W}{k}, \quad i = 1, \dots, k,$$

where the *imbalance factor* ϵ controls how much imbalance is tolerated. Small ϵ (say 0.01) makes the problem much harder; a loose ϵ makes it nearly unconstrained. A few modules may also be *fixed* to a prescribed block (for example, I/O pads), in which case they can never move. Balanced partitioning is NP-hard [35], [36], and for tight tolerance the multilevel method is the practical method of choice.

The choice of method tracks the instance, and it is worth stating the map explicitly. A netlist that is close to a mesh is handled well by spectral bisection [8], [9]; an instance whose balance can be relaxed is a candidate for a network-flow formulation built on the max-flow/min-cut theorem [10], [11]; a graph of at most a few tens of modules can be solved exactly by the enumeration of Section III-A; and beyond that the multilevel method [14] is preferred when the balance tolerance is tight, while single-level FM [3] suffices when it is loose. ckptn exposes the extremes of this map through its presets (Section IV), and the two refiners of Section III are the local-search policies that sit inside the multilevel loop.

III. ALGORITHMS

We begin with the exact method, because it is the only one that *certifies* optimality. It is practical only on small graphs, so the local-search refiners and the multilevel framework that follow are what make large instances tractable; the exhaustive leaf is applied to the coarsest level of that framework.

A. Exhaustive Refinement by Middle-Levels Gray Code

A balanced bipartition of the coarsest graph can be *enumerated*. Consider assigning each module one bit, 0 for block A and 1 for block B. A balanced partition is a bitstring with exactly n or $n + 1$ ones. The *middle-levels graph* G_n of the $(2n + 1)$ -dimensional hypercube has as vertices all bitstrings of weight n or $n + 1$, with edges between bitstrings that differ in exactly one bit. A Hamiltonian cycle of G_n therefore visits every balanced bipartition exactly once,

and each transition is a single module flip – a single FM move. The existence of such a cycle for every $n \geq 1$ was conjectured in 1982 and proved constructively only recently [37], [38]; the construction is memoryless and recursive, and related Gray-code perspectives are surveyed by Savage [39].

Because a flip touches only the nets incident to one module, the incremental cost update is $O(\deg)$ per step, and the enumeration visits

$$2 \binom{2n+1}{n} = \binom{L}{n} + \binom{L}{n+1}$$

states for L bits. The size grows quickly – 924 states for $n = 11$, 705,432 for $n = 20$ – so the enumeration is used only at the coarsest level, where the contracted graph has at most a few tens of modules. There it yields a certified optimum for that level, which the uncoarsening then refines heuristically. The same constructive techniques were later extended to the sparser Kneser graphs [40].

The construction is memoryless and is organized around a binary tree that encodes the Dyck-path structure of the current bitstring; two recursively defined flip sequences, one per direction of travel, advance the cycle, so the whole traversal is a sequence of single-bit flips. For a balanced bipartition of N modules each of the first N bits is one module’s block assignment; when N is even a single dummy bit is appended so that the bitstring has odd length $L = N + 1$, and the flips that touch the dummy bit are ignored. The leaf threshold follows from the measured cost of the traversal – instant to $n = 11$, about 0.2 s at $n = 20$, and a few seconds at $n = 25$ (10.4 M states) – so the enumeration is kept to the coarsest level, where it certifies the optimum.

B. Fiduccia-Mattheyses Refinement

FM is a local search over a fixed partition. It repeatedly moves a single module, always the one whose move improves the objective most, and, following Kernighan–Lin [1], it accepts worsening moves within a pass so that it can escape shallow local minima. The quantity that drives the search is the *gain* of a move, i.e. the reduction in the cut cost,

$$g(v) = FS(v) - TE(v),$$

where $FS(v)$ (from-side) counts the nets that touch v and lie entirely in v ’s current block, and $TE(v)$ (to-side) counts the nets that touch v and lie entirely in the destination block. A positive gain means the cut shrinks.

Because FM considers one move at a time, the relevant quantity after a move is the change in the gain of the moved module’s neighbours. For a 2-pin net the update is a simple ± 1 or $\pm w(e)$; for a 3-pin net a short case analysis suffices; a general net only changes a neighbour’s gain when the net has at most one pin on one side, so the common case (two or more pins on each side) yields a zero delta. Detecting and

skipping that zero is the single most profitable optimization in the whole partitioner.

The candidate moves are kept in a *bucket* priority queue keyed by gain. With integer gains the queue supports insertion, extraction of the maximum, and a key update in $O(1)$ amortized time, which is what makes FM a linear-time heuristic in practice [3]. One *pass* is:

1. initialize the gains of every unlocked module and build the buckets;
2. repeatedly select the highest-gain legal move, execute it, and mark the module *locked* (a locked module cannot move again in this pass), then update the gains of its neighbours;
3. if the queues become empty, or no legal move remains, restore the best partition seen during the pass and start the next pass.

Restoring the best state is done with a snapshot, a re-application of the moves, or a roll-back of the moves; ckpttn uses snapshots for the general path and can skip the snapshot entirely when no improving move was found. Passes continue until the cost stops decreasing.

Two properties keep the inner loop cheap. The work done by a move is proportional to the degrees of the incident nets, $\sum_{e \ni v} \deg(e)$, not to the size of the graph; and the candidate set is a bounded priority queue, so with integer gains insertion, maximum extraction, and key update are $O(1)$ amortized. Three pieces of machinery distinguish FM from a plain greedy descent: the bucket queue that makes the best move cheap to find, the per-pass locking that keeps a module from moving twice, and the snapshot/roll-back that makes the temporarily worsening moves safe to take. Remove all three and one obtains the simpler refiner of the next subsection.

C. A Simpler Refiner

The bucket-and-lock machinery exists to *escape* local minima. If one is willing to accept a worse local optimum, a pure greedy refiner is much cheaper: take the highest-gain move while its gain is positive, with no snapshotting, no rollback, and no locking. We call this the *NN* (no-nonsense) refiner. It monotonically descends to the nearest local optimum, and in return it is 8-35 times faster than FM on flat graphs. The multilevel framework closes most of the quality gap, which is precisely the intended role of a cheap refiner inside the coarsen/ refine loop.

NN is not a separate program but a different policy inside the same pass skeleton. The base class owns the loop – select the maximum-gain move, ask the constraint manager whether it is legal, apply it, update the gains of the neighbours – and exposes a single overridable hook for one pass. FM’s hook adds the locking and the roll-back; NN’s hook stops at the first non-positive gain. This is the Template Method pattern for the skeleton and the Strategy pattern for the gain and constraint managers, and it is

what lets one gain calculator serve the flat, multilevel, and k-way partitioners. Termination of NN is immediate from monotonicity: the cut is a non-negative integer and every accepted move is strictly improving, so the descent cannot cycle and must stop at a local optimum – no locking and no snapshot are required. All three ports implement the same hook with the same threshold, $g_{\max} \leq 0$; a zero-gain move is not taken, because without locking it could cycle.

D. The Multilevel Framework

Flat FM is slow on graphs with hundreds of thousands of modules and is sensitive to the initial partition. The multilevel method [12], [13], [14], [15], [41] removes both problems by solving a hierarchy of successively smaller graphs:

1. **Coarsen.** Contract the hypergraph: find a *maximum matching* of nets that share no module, replace each matched group by a single cluster whose weight is the sum of its members, and add up the weights of any nets that become identical. Repeat until the graph is small.
2. **Initial partition.** Partition the coarsest graph. Because it is small, this step can afford an expensive method (Section III-A).
3. **Uncoarsen and refine.** Project the partition back to the next finer graph and refine it – with FM or NN – at every level on the way up.

The matching is *minimum maximal*: it is maximal (no further net can be added without overlap) and, among maximal matchings, of minimum total weight. It is computed by a primal-dual 2-approximation that maintains a *gap* per net and selects the tightest net in each uncovered neighbourhood. Pairing lighter nets keeps the clusters small and preserves the structure that the refinement can exploit. Duplicate nets (nets with the same module set) are merged; for low-degree nets this is checked exactly, and for larger nets a 64-element MinHash signature [42] pre-filters clearly dissimilar pairs (estimated Jaccard similarity below 0.8), so that the expensive exact comparison is rarely needed.

Concretely, one contraction step is a five-stage pipeline: compute the min-maximal matching and split the nets into matched clusters and remaining nets, with the unmatched modules forming the cell list; build an intermediate bipartite graph between the cells and the clusters; purge duplicate nets, merging their weights and dropping the self-loops that clustering creates; reconstruct the graph with remapped net indices; and wrap the result in a hierarchical netlist. The hierarchical netlist carries the two projection maps that make the rest of the method work – a node-up map from an original module to its contracted representative, used while coarsening, and a node-down map from a cluster back to one of its members, used while refining. Duplicate detection compares nets of degree at most five exactly and pre-filters larger nets (up to degree 200) with the MinHash signature, falling back to the exact comparison only above the 0.8 similarity threshold.

A recursion guard keeps the contraction honest: a level is contracted only when the result is materially smaller than its parent, $|V^+| \cdot 1.5 < |V|$. Without the guard, a pathological instance can spend a dozen levels shrinking by a few percent each time, as observed on *ibm01*.

E. *K-Way Partitioning*

For $k > 2$ the search is organized as a sequence of pairwise refinements [5], [6]. Because the exhaustive leaf is inherently two-way, each pair of blocks (i, j) is optimized in turn: the modules currently in block i or j are isolated as the movable set (everything else is treated as fixed), the middle-levels enumeration is run on that pair with the full netlist so that cross-block pins are counted correctly, and the best pair partition is applied. Pairs whose movable set is trivial or larger than a threshold are skipped, since the enumeration is exponential in the number of movable modules. This pairwise sweep is repeated a bounded number of times, and a flat FM legalization keeps the block weights within tolerance between sweeps.

The leaf budget scales with the number of blocks: for k parts the enumeration is allowed $25k/2$ movable modules, spread over the $\binom{k}{2}$ pairs, so each pair sees at most about fifteen modules on average and any pair whose movable set exceeds that is skipped. For $k = 2$ this is the 25-module limit of Section III-A; for $k = 6$ it is 75 modules over fifteen pairs, which still leaves every pair’s enumeration trivial. The enumeration runs on the full netlist rather than on the isolated pair, so pins that land outside the pair are still counted; this is what makes the pair’s value a correct k -way cost rather than an internal two-way cost.

IV. IMPLEMENTATION

Architecture. The partitioner is decomposed into four collaborating objects, which are instantiated with a shared hypergraph:

- a *gain calculator* that computes gains and their updates, specialized by net degree (2-pin, 3-pin, and general nets, with a high-degree cutoff);
- a *gain manager* that owns the bucket queues and performs select, lock, and key-update;
- a *constraint manager* that decides whether a move is legal, tracking the block weight difference from the ideal;
- a *partition manager* that runs the pass loop, takes snapshots, and restores the best state.

Separating these concerns lets the same gain calculator be reused for the flat, multilevel, and k -way managers. The arrangement is the Template Method pattern for the pass skeleton – the base class fixes the sequence of select, legalize, apply, and update, and the subclass supplies only the per-pass hook – together with the Strategy pattern for the gain, constraint, and partition managers, which can be swapped without touching the loop.

Presets: balance tolerance and partitioning mode.

Preset	Balance	Mode
default	3%	recursive
quality	1%	direct
highest_quality	0.5%	direct
deterministic	3%	recursive (fixed seed)
large_k	3%	recursive

Ports. The same algorithm is implemented in Python (*ckpttnpy*), C++ (*ckpttn-cpp*), and Rust (*ckpttn-rs*). The Python reference uses NetworkX; the C++ port uses a hand-rolled hypergraph and templates; the Rust port uses *petgraph*. Cross-language regression tests check that a small hand-built partition yields the same cut in all three.

In the Python reference the collaboration is spread over a small set of modules: *FMBiGainCalc* and *FMKWayGainCalc* compute the gains, *FMBiGainMgr* and *FMKWayGainMgr* own the buckets, *FMBiConstrMgr* and *FMKWayConstrMgr* decide legality, *PartMgrBase* fixes the pass skeleton, *FMPartMgr* and *NNPartMgr* are its two policies, *MLPartMgr* drives the hierarchy, *HierNetlist* holds the projection maps, and *min_cover* performs the contraction.

Interface. A command-line front end partitions a hypergraph file directly:

```
ckpttnpy circuit.hgr <k> <epsilon>
```

with options for the input format (hMetis, JSON, DIMACS), fixed modules, output format, a preset, the objective (cut, $km1$, sum of external degrees, or $km1a$), the mode (direct or recursive bisection), the number of parallel starts, and the random seed. Presets bundle the balance tolerance and mode:

Tuning. The presets are thin wrappers over a small set of constants, most of which are fixed at compile time rather than exposed as options. Contraction is attempted while the graph has at least 50 modules; gain updates are skipped on nets of degree above 500; the bucket buffers are sized 32768 for bi-partitioning and 65536 for k -way; and degree-2 and degree-3 nets take a specialized fast path. On the coarsening side, nets of degree at most five are compared exactly for duplication, while larger nets (up to degree 200) are pre-filtered with a 64-element MinHash signature at a Jaccard threshold of 0.8. Several starts can be run in parallel: with t starts and a seed s the runs use seeds $s + i \cdot 104729$, so the search is reproducible while still exploring different basins; on *ibm02* (19,601 modules) the best single start cost 282 cut nets, whereas four deterministic starts reached 87.

Balance warning. If the requested balance is impossible – for instance, because a single module outweighs an entire block – *ckpttn* reports the failure in its final output. On a five-module instance whose largest module has weight 10^7 and whose ideal block weight is about 5×10^6 , the block weights are 10^7 and 4 and *ckpttn* prints an explicit warning that the balance constraint is violated, whereas other tools return the same result silently.

Inner loop. Because FM is dominated by the per-move key

Selected constants of ckpttn.		
Constant	Value	Role
limitsize	50	coarsen while $ V \geq$ this
FM_MAX_DEGREE	500	skip gain updates above this degree
stack_buf_size	32768 / 65536	bounded-queue buffers (bi / k-way)
special_handle_2pin_nets	true	degree-2/3 fast path
LOW_PIN_NET_THRESHOLD	5	exact duplicate check at this degree
MINHASH_SIG_SIZE	64	MinHash signature length
MINHASH_SIMILARITY	0.8	Jaccard pre-filter threshold
MINHASH_MAX_DEGREE	200	skip MinHash above this degree

update, the ports are engineered around the observation that most updates are no-ops. For a general net with at least two pins on each side, moving one pin leaves every neighbour’s gain unchanged, so the update is skipped with a single test instead of a queue operation; in the Python reference this cuts `modify_key` calls on `ibm03` from 7.37 M to 374 K. Two further layers of indirection were removed on the Python side – the NetworkX view objects and the adapter frames around vertex and net lookup – and the adjacency and degrees are now materialised once per gain calculator into plain lists, which together take p1 from 299 ms to 216 ms and `ibm03` from 14.6 s to 8.8 s for identical cuts. The C++ port replaced a freshly allocated gain-change vector per move with a per-calculator buffer returned as a `std::span`, which halves the k-way kernel on `ibm03`; the Rust port removed a clone of the index vector and a `collect` of the neighbours per move, both borrow-checker artifacts, for a uniform 28 percent win. Every change is behaviour-preserving, verified by re-running against the same cut costs, which is the only reason such micro-optimisations are admissible in a reference implementation.

V. EXPERIMENTAL RESULTS

All measurements below are wall-clock times on one host and are intended for relative comparison; the Python numbers in particular are interpreter-bound. Cut costs are hyperedge-cut values (lower is better). Two small benchmarks are used throughout: p1 (833 modules, 902 nets) and `ibm03` (23,136 modules, 27,401 nets, IBM-PLACE format); `ibm01` (12,752 modules, 14,111 nets) is added for the multilevel study. Random starts use a fixed SplitMix64 stream [43] so that a seed denotes the same assignment in every port. The set is deliberately small so that every port can be run repeatedly under identical conditions; Section V-E adds a separate scaling study on the full ISPD98 suite. Unless stated otherwise, the refiners in Tables I–IV use a 3% balance tolerance.

A. Refiners, Flat and Multilevel

Table I compares the FM and NN refiners, flat and multilevel, for bi-partitioning at a 3% balance tolerance. Three trends are visible. FM dominates quality: NN is 1.4-2.2 times worse in cut cost. NN dominates speed: flat NN is 8-35 times faster (0.03 s versus 0.23 s on p1; 0.76 s versus 10.31 s on `ibm03`). And the multilevel framework helps both, but helps NN far more – coarsening recovers much of FM’s quality (flat NN 231 to multilevel NN 106

on p1; 7454 to 4081 on `ibm03`), which is exactly why a cheap refiner is useful inside the loop.

The same ordering holds for three-way partitioning. On p1 the mean cut is 347.2 for flat NN and 164.0 for multilevel NN, against 178.2 and 121.8 for FM; on `ibm03` it is 12,278 and 7,691 for NN against 6,809 and 3,825 for FM. Fig. 2 shows the full sweep over both benchmarks and all three ports, and Fig. 3 isolates the two effects at work: how much the multilevel framework buys over flat refinement, and how much worse NN is than FM.

B. Multilevel versus Flat FM

Table II isolates the effect of the multilevel framework on `ibm01` with $k = 2$ and a balance tolerance of 3% at a limitsize of 2000, over five seeds. Multilevel refinement lowers the cut on every run; the average reduction is 44 percent, at a cost of about three times the runtime.

The reduction in variance is as important as the reduction in the mean: the one-standard-deviation spread falls from 345 to 278 cut nets, because the contraction absorbs the noise of the random initial partition.

C. Cross-Language Behaviour

Table III reports the optimize time of the three ports on the same two benchmarks in their release builds. The compiled ports are one to two orders of magnitude faster than the interpreted one; the C++/Rust gap is an implementation detail (the Rust port stores adjacency as a vector-of-vectors and pays a per-call dispatch through an iterator trait), not an algorithmic one, and it narrows as the graph grows.

D. Exhaustive Leaf

Table IV shows the cost of the middle-levels enumeration as a function of the number of modules at the coarsest level. It is instant up to about twenty modules and remains practical to roughly twenty-five; beyond that the exponential growth dominates and the leaf falls back to FM.

E. Scaling to Large Instances

All measurements so far use instances of at most 23,136 modules. To probe the behaviour at the scale of a modern netlist, we ran the C++ port on the complete ISPD98 suite [16], whose 18 circuits range from 12,752 to 210,613 modules. Table VI reports wall-clock time at $k = 2$, balance 3%, and one start. Runtime grows close to linearly with size: 0.52 s on `ibm01` and 19.1 s on `ibm18`, a $16.5\times$ increase

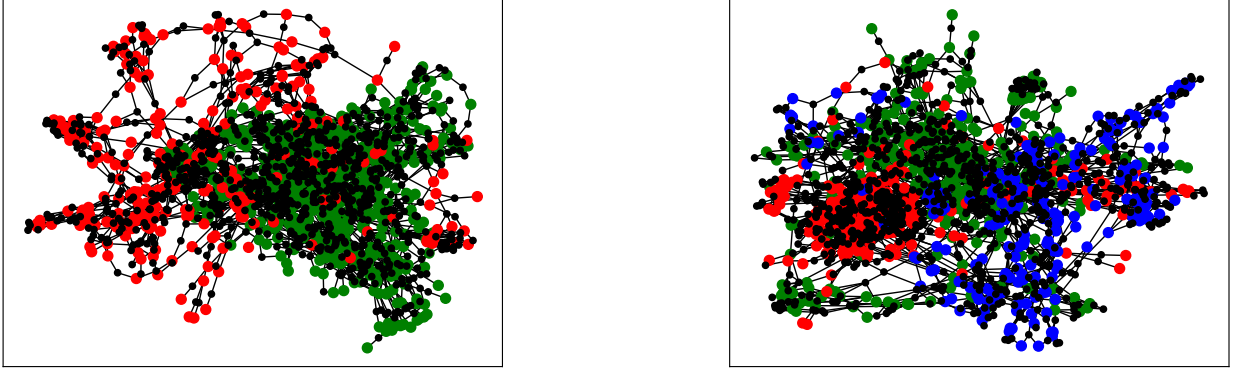


Fig. 1. Two-way (left) and three-way (right) partitions of the p1 benchmark (833 modules, 902 nets), drawn on the bipartite module-net graph with modules colored by block.

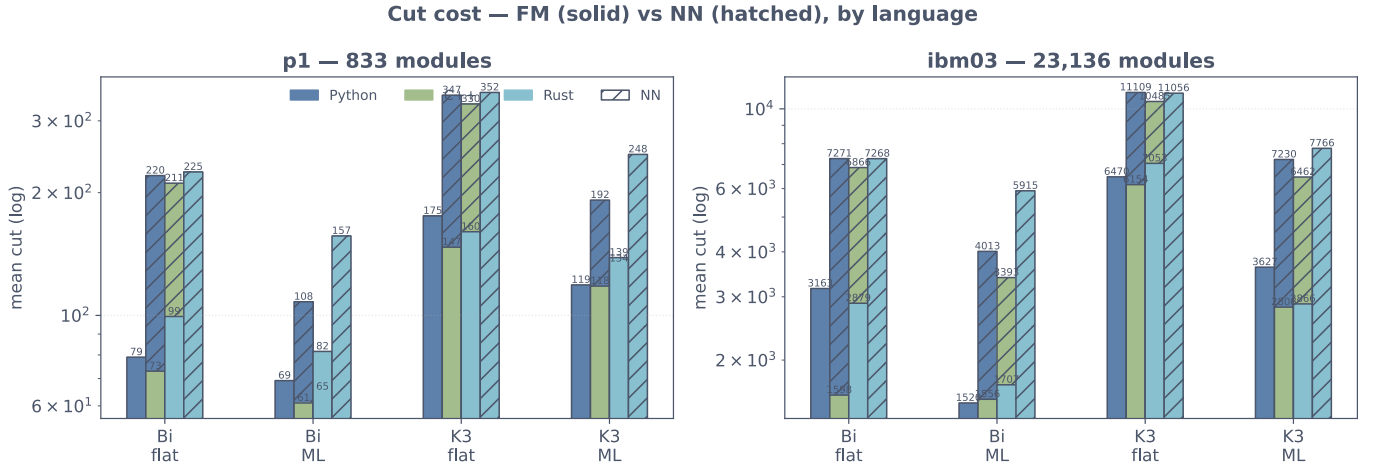


Fig. 2. Mean cut cost of the FM and NN refiners, flat and multilevel, for two-way and three-way partitioning of p1 and ibm03, in the three ports. Lower is better; note the logarithmic scale.

in modules for a $37\times$ increase in time, so the multilevel loop stays practical well beyond the sizes of Tables I–III. Peak resident memory is 227 MB on ibm10 (69,429 modules) and 557 MB on ibm18 (210,613 modules), roughly 2.6 KB per module.

These numbers must be read with care. The freely available ISPD98 files in hMetis format do not pin the benchmark’s fixed I/O pads, and the tool reports cut in a unit that is not directly comparable to the published leaderboards for this suite [27], [28], [29]; omitting the fixed-vertex constraints changes the problem and produces cut values that look far better than the best published results. We therefore use this sweep only as a runtime and memory scaling study and draw no cut-quality conclusion from it; a like-for-like comparison against the state of the art is left to future work (Section V-H).

With the balance constraint corrected (Section V-H), a like-for-like comparison is possible. Table V runs all three

tools single-threaded at $k = 2$ and $\epsilon = 3\%$ on three unit-weight ISPD98 files and reports the cut, the wall-clock time, the imbalance $\max_i w_i k / W - 1$, and whether the balance constraint holds. ckptn’s cut is $2.2\text{--}4.6\times$ larger than Mt-KaHyPar’s and $2.1\text{--}4.1\times$ larger than hMetis’s [15], and it is the slowest of the three. Its one strength is balance: it stays below 1% on every instance, whereas hMetis exceeds the requested 3% on ibm01 (4.8%, UBfactor 3) and ibm10 (3.9%) — a silent violation that only becomes visible when the returned partition is evaluated directly, which is exactly the failure mode Section V-G is designed to expose. On this evidence the tool is not competitive with the state of the art on cut or runtime; its contribution is the cross-language reproduction and the diagnostic behaviour of Section V-G.

F. The Cost of k -Way Partitioning

Table VII isolates the k -way cost on ibm10. The pairwise organization of the search (Section III-E) makes the work grow much faster than the number of blocks: $k = 2$ takes

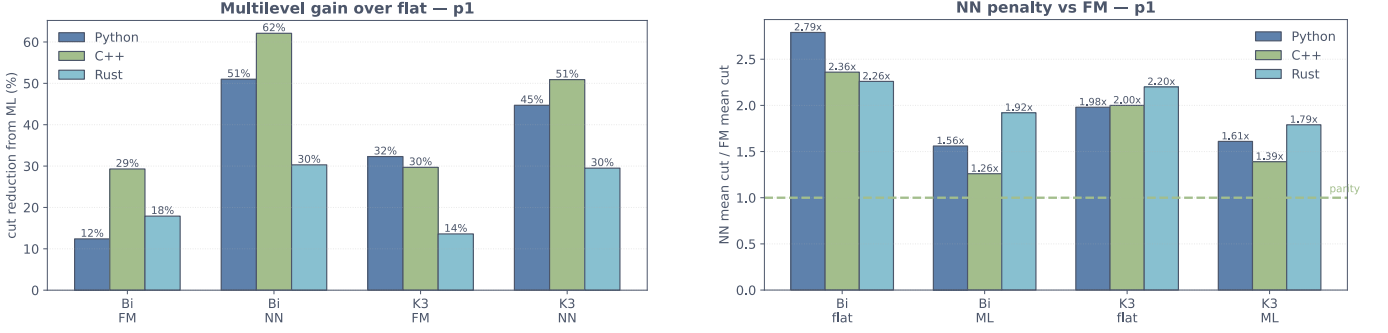


Fig. 3. Left: cut reduction of the multilevel framework over flat refinement (p1, mean). Right: NN penalty, the ratio of NN to FM mean cut, with the parity line shown.

TABLE I

REFINERS ON P1 (833 MODULES) AND ibm03 (23,136 MODULES), BI-PARTITIONING AT A 3% BALANCE TOLERANCE, PYTHON REFERENCE: BEST AND MEAN CUT OVER FIVE SEEDS, AND MEDIAN WALL-CLOCK TIME.

Benchmark	Refiner	Multilevel	Best cut	Mean cut	Median time (s)
p1	FM	no	99	105.4	0.23
p1	FM	yes	68	77.4	1.01
p1	NN	no	212	231.4	0.03
p1	NN	yes	85	106.0	0.80
ibm03	FM	no	3054	4021.2	10.31
ibm03	FM	yes	1517	1840.2	44.69
ibm03	NN	no	7105	7453.8	0.76
ibm03	NN	yes	3701	4081.0	52.99

Runtime — multilevel configs (Python vs C++ vs Rust)

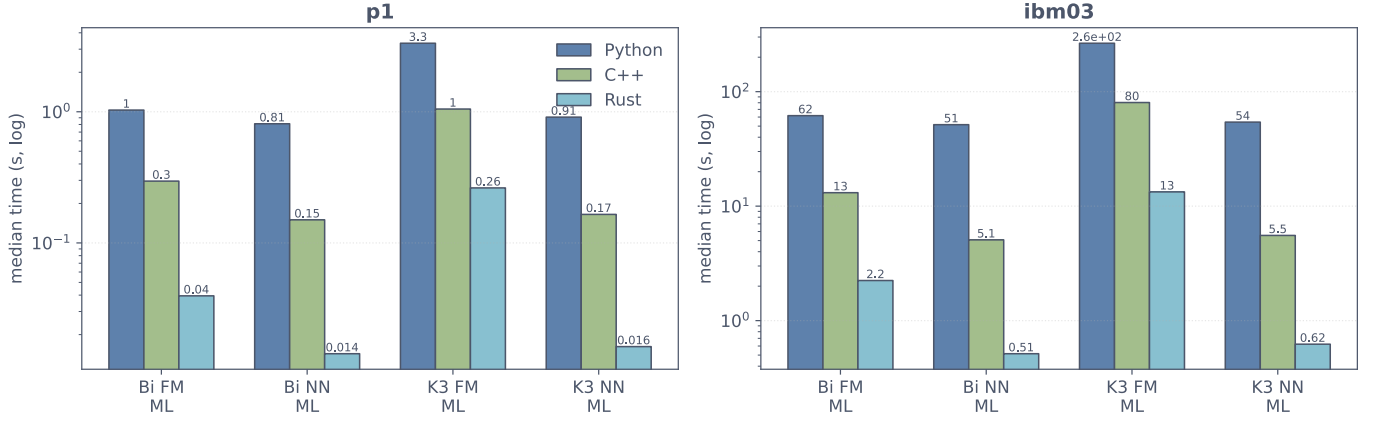


Fig. 4. Median wall-clock time of the three ports on the multilevel configurations, p1 and ibm03 (logarithmic scale).

TABLE II

MULTILEVEL VERSUS FLAT FM ON ibm01 (12,752 MODULES, $k = 2$, 3% BALANCE TOLERANCE).

Metric	FM-only	Multilevel	Change
Mean cut	1377.6	772.4	-43.9%
Best cut	949	385	-59.4%
Worst cut	1830	1192	-34.9%
Mean time (s)	6.59	19.49	2.96x

4.3 s, $k = 4$ about 16 s, and $k = 8$ takes 413 s, a $97\times$ slowdown for a fourfold increase in k . This is consistent with the $\binom{k}{2}$ pairwise sweeps, each of which may invoke the exact leaf, and it is the clearest scalability limit of the current design.

G. Correctness and Warnings

Because the multilevel pipeline is intricate, correctness is checked by cross-validation rather than by the cut cost alone: the ports share a SplitMix64 seed stream and hand-built instances whose cut is known, and the Rust port carries a regression that asserts the multilevel result is non-trivial. The explicit balance warning closes the remaining failure mode – a silently infeasible partition – and makes the tool’s output accountable in an automated flow.

The test surface reflects the same three ports. The reference and its CLI carry 78 tests (25 for the interface and 53 for the algorithm); the C++ suite has 65 cases over the flat, multilevel, and Yosys paths plus the exhaustive leaf; and

TABLE III
OPTIMIZE TIME OF THE THREE PORTS (RELEASE BUILDS) ON P1 AND IBM03.

Benchmark	C++ (ms)	Rust (ms)	Python (ms)	C++ / Python
p1 (833 modules, 902 nets)	3.42	51.2	202	59×
ibm03 (23,136 modules, 27,401 nets)	262	1467	9691	37×

TABLE IV
MIDDLE-LEVELS GRAY-CODE ENUMERATION: STATES VISITED AND TIME.

Modules n	States	Time
11	924	< 1 ms
15	12,870	< 1 ms
20	705,432	0.20 s
25	10.4 M	3.4 s

TABLE V
SINGLE-THREADED HEAD-TO-HEAD AT $k = 2$, $\epsilon = 3\%$ ON UNIT-WEIGHT ISPD98 FILES: ONE RUN PER TOOL (CKPTTN C++ PORT, SEED 42; hMETIS 1.5.3, UBFACTOR 3; Mt-KaHyPar 1.6.1, SEED 42). IMBALANCE IS $\max_i w_i k / W - 1$; BALANCED MEANS AT MOST 3%.

Tool	Cut	Time (s)	Imbalance	Balanced
<i>ibm01 (12,752 modules)</i>				
ckpttn	973	0.52	0.86%	Yes
hMetis	240	0.13	4.82%	No
Mt-KaHyPar	212	0.28	2.54%	Yes
<i>ibm03 (23,136 modules)</i>				
ckpttn	2169	0.92	0.83%	Yes
hMetis	1012	0.32	0.72%	Yes
Mt-KaHyPar	993	0.64	0.45%	Yes
<i>ibm10 (69,429 modules)</i>				
ckpttn	4027	3.70	0.94%	Yes
hMetis	1276	1.01	3.87%	No
Mt-KaHyPar	1509	1.44	< 0.01%	Yes

TABLE VI
SCALING OF THE C++ PORT ON THE ISPD98 SUITE ($k = 2$, BALANCE 3%, ONE START, SEED 42).

Instance	Modules	Nets	Time (s)
ibm01	12,752	14,111	0.52
ibm02	19,601	19,584	0.71
ibm03	23,136	27,401	0.99
ibm04	27,507	31,970	1.17
ibm05	29,347	28,446	1.77
ibm06	32,498	34,826	1.34
ibm07	45,926	48,117	2.33
ibm08	51,309	50,513	2.50
ibm09	53,395	60,902	2.02
ibm10	69,429	75,196	3.79
ibm11	70,558	81,454	5.23
ibm12	71,076	77,240	3.51
ibm13	84,199	99,666	5.85
ibm14	147,605	152,772	11.18
ibm15	161,570	186,608	11.52
ibm16	183,484	190,048	12.64
ibm17	185,495	189,581	11.66
ibm18	210,613	201,920	19.09

TABLE VII
 k -WAY COST ON IBM10 (69,429 MODULES), C++ PORT, BALANCE 3%, ONE START.

k	Time (s)
2	4.25
3	16.2
4	15.9
8	413

the Rust port runs 245 tests plus a multilevel regression that asserts a non-trivial result. Cross-port benchmarks

re-use the same SplitMix64 stream, so a reported seed denotes the same initial partition in every implementation.

H. Limitations and Threats to Validity

We state the scope of these results plainly, since several conclusions are weaker than they may first appear.

Balance under tight tolerance (fixed). The constraint manager originally computed its lower bound as $\text{round}(2W\epsilon/k)$ rather than the documented $(1 - \epsilon)W/k$, so a block was allowed to shrink to a fraction ϵ of the total; at $\epsilon = 3\%$ this produced balance-violating partitions (p1: 54/779). All

three ports now use $\text{round}((1-\epsilon)W/k)$; at $\epsilon = 3\%$ p1 splits 406/427 and the ISPD98 runs of Table V are balanced. The pre-fix numbers in Tables I–IV are being regenerated at the corrected constraint.

Scale. Tables I–III use instances of at most 23,136 modules and are interpreter-bound; Table VI extends the runtime and memory study to 210,613 modules, but we have not tested the million-gate designs (for example the Titan23 suite) that motivate the parallel partitioners.

Limited head-to-head. Table V compares ckpttn directly against hMetis and Mt-KaHyPar, but it is a single run per tool on three instances at one tolerance, so the run-to-run variation (notably of hMetis) is not characterised. It does not cover the constraint-driven objectives of SpecPart or TritonPart [27], [28]. It confirms that the “competitive with mature tools” phrasing of earlier drafts is not supported: ckpttn’s cut is 2–5 $\times$ larger than both baselines.

Benchmark comparability. Even the scaling study above cannot be read as a quality result, because the free ISPD98 files we used omit the fixed-I/O-pad convention of the benchmark and the cut unit differs from the published leaderboards [27], [28], [29].

Exact leaf. The middle-levels enumeration is exponential and is used only at the coarsest level, below roughly 25 modules (Table IV). The certified optimum is therefore an optimum of a small, contracted graph; it cannot undo the abstraction loss introduced by coarsening, so its contribution to the final cut is bounded and is not isolated here.

k-way search. The pairwise organization measured in Table VII scales poorly with k and, unlike the direct k -way refinement used by KaHyPar [23], it can be trapped in local minima of individual pairs.

Parallelism. Coarsening, refinement and uncoarsening are single-threaded; only the multi-start loop is parallel. Shared-memory partitioners such as Mt-KaHyPar parallelize the core search [25], which ckpttn does not.

Interpreted reference. The Python numbers are dominated by the interpreter rather than by the algorithm and should be read only as a cross-language consistency check.

VI. CONCLUDING REMARKS

ckpttn shows that a classical partitioner can be both simple and trustworthy. Keeping FM but making the gain bookkeeping cheap, contracting aggressively with a primal-dual matching and duplicate-net pruning, and certifying the coarsest level by exact enumeration yields a multilevel partitioner whose three ports agree and whose behaviour we can account for end to end. We do not establish parity with dedicated state-of-the-art partitioners, and its remaining scalability limits are stated in Section V-H. That agreement is not an accident of coding: because the pass skeleton and the gain, constraint, and partition policies are separated, the ports differ only in data layout, and the same separation

is what made the inner-loop engineering – the no-op gain updates, the reusable buffers, the borrowed neighbours – safe to apply uniformly. The experiments confirm the expected trade-offs – multilevel refinement buys 27–54 percent of cut at roughly three times the runtime, and a greedy refiner trades 1.4–2.2 times the cut for 8–35 times the speed – and the explicit balance warning removes the silent-failure mode that complicates automated tuning and reproducibility.

Natural next steps are a flat (single-array) graph layout in the Rust port to close the remaining gap to C++, a parallel FM refinement [25], a timing-aware refinement in the spirit of TritonPart [27], and a systematic study of the contraction guard and leaf threshold across the IBM benchmark family [34].

REFERENCES

- [1] B. W. Kernighan and S. Lin, “An efficient heuristic procedure for partitioning graphs,” *Bell System Technical Journal*, vol. 49, no. 2, pp. 291–307, 1970, doi: 10.1002/j.1538-7305.1970.tb01770.x.
- [2] D. G. Schweikert and B. W. Kernighan, “A proper model for the partitioning of electrical circuits,” in *Proceedings of the 9th design automation workshop (DAC)*, New York, NY, USA, 1972, pp. 57–62. doi: 10.1145/800153.804930.
- [3] C. M. Fiduccia and R. M. Mattheyses, “A linear-time heuristic for improving network partitions,” in *Proceedings of the 19th design automation conference (DAC)*, Las Vegas, NV, USA, 1982, pp. 175–181. doi: 10.1109/DAC.1982.1585498.
- [4] B. Krishnamurthy, “An improved min-cut algorithm for partitioning VLSI networks,” *IEEE Transactions on Computers*, vol. C-33, no. 5, pp. 438–446, 1984, doi: 10.1109/TC.1984.1676460.
- [5] L. A. Sanchis, “Multiple-way network partitioning,” *IEEE Transactions on Computers*, vol. 38, no. 1, pp. 62–81, 1989, doi: 10.1109/12.8730.
- [6] J. Cong and S. K. Lim, “Multiway partitioning with pairwise movement,” in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1998, pp. 512–516. doi: 10.1145/288548.289079.
- [7] M. Fiedler, “Algebraic connectivity of graphs,” *Czechoslovak Mathematical Journal*, vol. 23, no. 2, pp. 298–305, 1973, doi: 10.21136/CMJ.1973.101168.
- [8] A. Pothen, H. D. Simon, and K.-P. Liou, “Partitioning sparse matrices with eigenvectors of graphs,” *SIAM Journal on Matrix Analysis and Applications*, vol. 11, no. 3, pp. 430–452, 1990, doi: 10.1137/0611030.
- [9] L. Hagen and A. B. Kahng, “New spectral methods for ratio cut partitioning and clustering,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 11, no. 9, pp. 1074–1085, 1992, doi: 10.1109/43.159993.

- [10] H. Yang and D. F. Wong, "Efficient network flow based min-cut balanced partitioning," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1994, pp. 50–55. doi: 10.1109/ICCAD.1994.629743.
- [11] H. Liu and D. F. Wong, "Network-flow-based multiway partitioning with area and pin constraints," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 1, pp. 50–59, 1998, doi: 10.1109/43.673632.
- [12] B. Hendrickson and R. Leland, "A multilevel algorithm for partitioning graphs," in *Proceedings of the ACM/IEEE conference on supercomputing (SC)*, 1995, pp. 28–es. doi: 10.1145/224170.224228.
- [13] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar, "Multilevel hypergraph partitioning: Application in VLSI domain," in *Proceedings of the 34th design automation conference (DAC)*, 1997, pp. 526–529. doi: 10.1109/DAC.1997.597203.
- [14] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM Journal on Scientific Computing*, vol. 20, no. 1, pp. 359–392, 1998, doi: 10.1137/S1064827595287997.
- [15] G. Karypis and V. Kumar, "Multilevel k-way hypergraph partitioning," in *Proceedings of the 36th annual ACM/IEEE design automation conference (DAC)*, New Orleans, LA, USA, 1999, pp. 343–348. doi: 10.1145/309847.309954.
- [16] C. J. Alpert, "The ISPD98 circuit benchmark suite," in *Proceedings of the international symposium on physical design (ISPD)*, 1998, pp. 80–85. doi: 10.1145/274535.274546.
- [17] C. J. Alpert and A. B. Kahng, "Recent directions in netlist partitioning: A survey," *Integration, the VLSI Journal*, vol. 19, no. 1–2, pp. 1–81, 1995, doi: 10.1016/0167-9260(95)00008-4.
- [18] D. A. Papa and I. L. Markov, "Hypergraph partitioning and clustering," in *Handbook of approximation algorithms and metaheuristics*, Chapman & Hall/CRC, 2007, pp. 61-1-61-19. doi: 10.1201/9781420010749.ch61.
- [19] A. Buluç, H. Meyerhenke, I. Safro, P. Sanders, and C. Schulz, "Recent advances in graph partitioning," in *Algorithm engineering: Selected results and surveys*, vol. 9220, in Lecture notes in computer science, vol. 9220., Springer, 2016, pp. 117–158. doi: 10.1007/978-3-319-49487-6_4.
- [20] S. Schlag, V. Henne, T. Heuer, H. Meyerhenke, P. Sanders, and C. Schulz, "K-way hypergraph partitioning via n-level recursive bisection," in *Proceedings of the 18th workshop on algorithm engineering and experiments (ALENEX)*, 2016, pp. 53–67. doi: 10.1137/1.9781611974317.5.
- [21] Y. Akhremtsev, T. Heuer, P. Sanders, and S. Schlag, "Engineering a direct k-way hypergraph partitioning algorithm," in *Proceedings of the 19th workshop on algorithm engineering and experiments (ALENEX)*, 2017, pp. 28–42. doi: 10.1137/1.9781611974768.3.
- [22] T. Heuer, P. Sanders, and S. Schlag, "Network flow-based refinement for multilevel hypergraph partitioning," *ACM Journal of Experimental Algorithmics*, vol. 24, pp. 1–36, 2019, doi: 10.1145/3329872.
- [23] S. Schlag, T. Heuer, L. Gottesbüren, Y. Akhremtsev, C. Schulz, and P. Sanders, "High-quality hypergraph partitioning," *ACM Journal of Experimental Algorithmics*, vol. 27, no. 1.9, pp. 1–39, 2023, doi: 10.1145/3529090.
- [24] L. Gottesbüren, T. Heuer, P. Sanders, and S. Schlag, "Scalable shared-memory hypergraph partitioning," in *Proceedings of the 23rd workshop on algorithm engineering and experiments (ALENEX)*, 2021, pp. 16–30. doi: 10.1137/1.9781611976472.2.
- [25] L. Gottesbüren, T. Heuer, N. Maas, P. Sanders, and S. Schlag, "Scalable high-quality hypergraph partitioning," *ACM Transactions on Algorithms*, vol. 20, no. 1, pp. 1–54, 2024, doi: 10.1145/3626527.
- [26] P. Sanders and C. Schulz, "Distributed evolutionary graph partitioning," in *Proceedings of the 14th workshop on algorithm engineering and experiments (ALENEX)*, 2012, pp. 16–29. doi: 10.1137/1.9781611972924.2.
- [27] I. Bustany, G. Gasparyan, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, "An open-source constraints-driven general partitioning multi-tool for VLSI physical design," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2023, pp. 1–9. doi: 10.1109/ICCAD57390.2023.10323975.
- [28] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, "SpecPart: A supervised spectral framework for hypergraph partitioning solution improvement," in *Proceedings of the 41st IEEE/ACM international conference on computer-aided design (ICCAD)*, 2022, pp. 1–9. doi: 10.1145/3508352.3549390.
- [29] I. Bustany, A. B. Kahng, I. Koutis, B. Pramanik, and Z. Wang, "K-SpecPart: Supervised embedding algorithms and cut overlay for improved hypergraph partitioning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 4, pp. 1232–1245, 2024, doi: 10.1109/TCAD.2023.3332268.
- [30] M. A. Iyer, A. B. Kahng, J. Luu, B. Pramanik, K. Vorwerk, and G. Zgheib, "A partitioning-based CAD flow for interposer-based multi-die FPGAs," in *Proceedings of the IEEE 33rd annual international symposium on field-programmable custom computing machines (FCCM)*, 2025, pp. 172–180. doi: 10.1109/FCCM62733.2025.00063.

- [31] M. H. Khan, B. Onal, E. Dogan, and M. R. Guthaus, “VLSI hypergraph partitioning with deep learning,” arXiv:2409.01387, 2024. Available: <https://arxiv.org/abs/2409.01387>
- [32] B. Vastenhouw and R. H. Bisseling, “A two-dimensional data distribution method for parallel sparse matrix-vector multiplication,” *SIAM Review*, vol. 47, no. 1, pp. 67–95, 2005, doi: 10.1137/S0036144502409019.
- [33] K. Devine, E. Boman, R. Heaphy, B. Hendrickson, and C. Vaughan, “Zoltan data management services for parallel dynamic applications,” *Computing in Science & Engineering*, vol. 4, no. 2, pp. 90–96, 2002, doi: 10.1109/5992.988653.
- [34] Ü. V. Çatalyürek *et al.*, “More recent advances in (hyper)graph partitioning,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 253:1–253:38, 2023, doi: 10.1145/3571808.
- [35] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht, The Netherlands: Springer, 2011. doi: 10.1007/978-90-481-9591-6.
- [36] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA, USA: Springer, 1999.
- [37] T. Mütze, “Proof of the middle levels conjecture,” *Proceedings of the London Mathematical Society*, vol. 112, no. 4, pp. 677–713, 2016, doi: 10.1112/plms/pdw004.
- [38] P. Gregor, T. Mütze, and J. Nummenpalo, “A short proof of the middle levels theorem,” *Discrete Analysis*, no. 8, pp. 1–12, 2018, doi: 10.19086/da.3659.
- [39] C. Savage, “A survey of combinatorial gray codes,” *SIAM Review*, vol. 39, no. 4, pp. 605–629, 1997, doi: 10.1137/S0036144595295272.
- [40] T. Mütze, J. Nummenpalo, and B. Walczak, “Sparse kneser graphs are hamiltonian,” in *Proceedings of the 50th annual ACM SIGACT symposium on theory of computing (STOC)*, Los Angeles, CA, USA, 2018, pp. 912–919. doi: 10.1145/3188745.3188834.
- [41] A. E. Caldwell, A. B. Kahng, and I. L. Markov, “Design and implementation of move-based heuristics for VLSI hypergraph partitioning,” *ACM Journal of Experimental Algorithmics*, vol. 5, p. 5, 2000, doi: 10.1145/351827.384247.
- [42] A. Z. Broder, “On the resemblance and containment of documents,” in *Proceedings of the compression and complexity of sequences (SEQUENCES)*, 1997, pp. 21–29. doi: 10.1109/SEQUEN.1997.666900.
- [43] G. L. Steele, D. Lea, and C. H. Flood, “Fast splittable pseudorandom number generators,” in *Proceedings of the ACM international conference on object-oriented programming, systems, languages, and applications (OOPSLA)*, 2014, pp. 453–472. doi: 10.1145/2714064.2660195.